

TEN GRID

HTML + CSS + JavaScript

게임 설명

전체 구조 개요

CSS 구조

구조 생성

렌더링

드래그 이벤트

타이머

게임 흐름

게임 설명

게임 시작

TEN GRID

게임 시작

게임 결과

게임 시작

GAME OVER

최종 점수

0

지운 셀

0칸

게임 화면

점수: 0

남은 시간 41s

9	8	6	5	2	8	8	5	7	3
2	4	5	8	6	7	3	4	2	2
9	9	9	9	7	6	6	8	2	4
1	2	5	6	2	6	3	6	9	3
7	9	1	7	4	8	9	4	2	5
4	5	6	3	3	6	1	6	1	4
8	2	1	3	9	1	7	7	3	3
5	6	5	9	8	4	3	2	9	6
4	1	7	3	8	1	3	7	2	7
1	8	9	3	8	1	9	8	9	1

1

게임 시작 시
게임 보드 생성

2

숫자 셀의 합이 10이 되도록
드래그 앤 드롭

3

숫자 합이 10이면 셀 삭제 &
지운 셀 * 10 점수 획득

4

타이머가 0이 되면
게임 종료

5

게임 시작 버튼과
최종 점수, 지운 셀 화면 출력

6

제한시간 내 최대한
많은 셀 제거가 목표

전체 구조 개요

HTML 구조

- 게임 이름
- 점수 / 타이머 표시
- 게임 시작 버튼
- 게임 보드 그리드
- 결과 출력 영역

CSS 스타일

- body / 레이아웃
- #game → 10열 그리드
- .cell 클릭 영역 35px
- .cell-inner 보이는 영역 30px
- .active / .empty 상태

JavaScript

- 전역 변수 선언
- 보드 생성 (createBoard)
- 화면 렌더링 (render)
- 이벤트 연결 (addEvents)
- 드래그 3단계 처리
- 타이머 관리
- 게임 시작 / 종료

CSS 구조 — 클릭 영역 vs 보이는 영역

```
#game {  
  grid-template-columns:  
    repeat(10, 35px);  
  /* 그리드 10개의 열 생성 */  
}  
  
.cell { /* 클릭 영역 */  
  width: 35px;  
  height: 35px; /* 꼭 찬 영역 */  
}  
  
.cell-inner { /* 보이는 영역 */  
  width: 30px;  
  height: 30px;  
  border-radius: 10px;  
  pointer-events: none;  
}
```

pointer-events: none → 이벤트는 .cell에서만 감지

.cell (35×35px, 클릭 영역)

.cell-inner
(30×30px, 보이는 영역)

← 5px 여백 (gap처럼 보임)

전역 변수

게임 전체에서 공유되는 상태값들

// 전역 변수 선언

```
let board = [];           // 10x10 숫자 배열 (게임판 상태)
let score = 0;            // 현재 점수
let time = 60;            // 남은 시간(초)
let timerInterval = null; // setInterval 핸들러

let isDragging = false; // 드래그 중 여부 플래그
let selected = [];       // 현재 선택된 셀 목록 [{r, c}, ...]
let sum = 0;             // 선택된 셀 숫자 합계
```

board

2차원 배열. board[r][c]에 1~9의 숫자를 저장. 셀이 사라지면 0으로 변경.

isDragging

마우스 버튼이 눌린 상태인지 추적. false이면 mouseenter 이벤트를 무시.

selected + sum

드래그 중 선택된 셀 좌표 목록과 합계. sum === 10이면 셀을 제거하고 점수 획득.

보드 생성 & 렌더링

createBoard()

```
board = [];  
for (let r = 0; r < 10; r++) {  
  let row = [];  
  for (let c = 0; c < 10; c++) {  
    let n = Math.floor(Math.random()*9)+1;  
    row.push(n); // 1~9 랜덤  
  }  
  board.push(row);  
}
```

이중 for 루프로 10×10 = 100개의 셀을 생성합니다.
Math.random()*9+1 → 1~9 사이 정수를 각 셀에 배정합니다.

render()

```
let html = '';  
for (r) for (c) {  
  if (board[r][c] === 0) {  
    html += '<div class=empty>...</div>'  
  } else {  
    html += '<div data-r=r data-c=c>숫자</div>'  
  }  
}  
game.innerHTML = html;  
addEvents(); // 이벤트 재연결
```

board 배열을 읽어 HTML 문자열을 조합 후 innerHTML로 주입.
data-r, data-c 속성으로 나중에 어느 셀인지 식별합니다.
상태가 바뀔 때마다 render()를 다시 호출해 화면을 동기화.

드래그 이벤트

mousedown → mouseenter → mouseup

1

startDrag() [mousedown]

```
if (empty) return;
isDragging = true;
selected = []; sum = 0;
selected.push({r, c});
sum += board[r][c];
cell.classList.add('active');
```

마우스 버튼을 누르는 순간 호출. 드래그 상태 초기화 후 첫 셀 선택.

2

dragging() [mouseenter]

```
if (!isDragging) return;
if (empty) return;
if (already) return; // 중복 방지
// 상하좌우 인접 체크
if (!isAdjacent) return;
selected.push({r,c}); sum += n;
```

마우스가 셀 위로 진입할 때 호출. 인접 & 미선택 셀만 추가. 대각선 불가.

3

endDrag() [mouseup]

```
isDragging = false;
if (sum === 10) {
  selected.forEach(cell => {
    board[cell.r][cell.c] = 0;
  });
  score++;
}
render();
```

버튼을 놓을 때 호출. 합이 10이면 셀을 보드에서 제거하고 점수 추가.

대각선 방지 알고리즘

마지막 선택 셀 기준, 상하좌우(거리=1)만 허용

```
let last = selected[selected.length - 1];

let dr = Math.abs(r - last.r); // 행 거리
let dc = Math.abs(c - last.c); // 열 거리

let isAdjacent =
  (dr === 1 && dc === 0) || // 상/하
  (dr === 0 && dc === 1); // 좌/우

if (!isAdjacent) return; // 대각선 차단
```

왜 이 조건인가?

- 상 : $r-1, c+0 \rightarrow dr=1, dc=0$ ✓
- 하 : $r+1, c+0 \rightarrow dr=1, dc=0$ ✓
- 좌 : $r+0, c-1 \rightarrow dr=0, dc=1$ ✓
- 우 : $r+0, c+1 \rightarrow dr=0, dc=1$ ✓
- 대각선 : $dr=1, dc=1 \rightarrow$ 조건 불만족 ✗

허용 (상하좌우)



★ = 현재 위치 ○ = 이동 가능 ● = 대각선 차단

타이머 & 게임 흐름

타이머 함수

```
// 시작
function startTimer() {
  timerInterval = setInterval(() => {
    time--;
    // 화면 업데이트
    if (time <= 0) endGame();
  }, 1000);
}

// 리셋
function resetTimer() {
  clearInterval(timerInterval);
  time = 60;
}
```

1. 게임 시작 버튼 클릭

2. startGame() 호출

3. score=0, createBoard(), render()

4. resetTimer() → startTimer()

5. 60초 카운트다운 시작

6. 유저 드래그 → sum=10이면 cell * 10의 score 획득

7. time === 0 → endGame()

8. 보드 제거, 최종 점수 표시

드래그 이벤트

mousedown → mouseenter → mouseup

1

startDrag() [mousedown]

```
if (empty) return;
isDragging = true;
selected = []; sum = 0;
selected.push({r, c});
sum += board[r][c];
cell.classList.add('active');
```

마우스 버튼을 누르는 순간 호출. 드래그 상태 초기화 후 첫 셀 선택.

2

dragging() [mouseenter]

```
if (!isDragging) return;
if (empty) return;
if (already) return; // 중복 방지
// 상하좌우 인접 체크
if (!isAdjacent) return;
selected.push({r,c}); sum += n;
```

마우스가 셀 위로 진입할 때 호출. 인접 & 미선택 셀만 추가. 대각선 불가.

3

endDrag() [mouseup]

```
isDragging = false;
if (sum === 10) {
  selected.forEach(cell => {
    board[cell.r][cell.c] = 0;
  });
  score++;
}
render();
```

버튼을 놓을 때 호출. 합이 10이면 셀을 보드에서 제거하고 점수 추가.



END

HTML + CSS + JavaScript